

CODSOFT – TASK 3

SECURITY CODE REVIEW REPORT

Project: Secure Notes

Technology: Python Flask + SQLite

1. Introduction

This report documents the security review of Secure Notes, a small educational web application developed using Python Flask and SQLite. The source code was examined for common security weaknesses, static analysis was performed with Bandit, manual review was completed, and security fixes were implemented in a separate secure version.

2. Objective

The objectives were to examine source code for security weaknesses, use a static analysis tool, perform manual security review, document vulnerabilities and recommended fixes, implement secure coding improvements, and compare results before and after remediation.

3. Application Description

Secure Notes provides user registration, login/logout, session-based authentication, personal note creation, note viewing and note deletion. It uses Flask for the web application and SQLite for local data storage.

4. Technology and Testing Environment

Programming language: Python

Framework: Flask

Database: SQLite

Static analysis: Bandit

Operating environment: Kali Linux

Testing scope: Local educational environment

Application: <http://127.0.0.1:5000>

5. Security Review Methodology

The intentionally vulnerable version was first reviewed with Bandit and manually inspected. Security improvements were then implemented in a separate secure version. The secure application was functionally tested and scanned again with Bandit.

6. Bandit BEFORE Results

The first scan identified three findings: B105 – hardcoded password/secret string (Low); B608 – hardcoded SQL expression / possible SQL injection vector (Medium); B201 – Flask debug mode enabled (High). The results were saved as bandit-before.txt and bandit-before.json.

7. Manual Code Review Findings

VULN-01 – Hardcoded Flask Secret Key – Medium: The vulnerable application stored the Flask secret key directly in source code. Recommendation: load secrets from protected environment variables or a secret-management system.

VULN-02 – SQL Injection Risk – High: The vulnerable login logic constructed SQL using user-controlled input. Recommendation: use parameterized SQL queries.

VULN-03 – Flask Debug Mode Enabled – High: Debug mode was enabled. Recommendation: disable debug mode in production.

VULN-04 – Plaintext Password Storage – High: Passwords were stored without secure hashing. Recommendation: use password hashing.

VULN-05 – Missing Input Validation – Medium: User input had limited validation. Recommendation: validate required fields, lengths and acceptable values.

VULN-06 – Missing CSRF Protection – Medium: State-changing forms lacked explicit CSRF protection. Recommendation: use CSRF protection in production.

VULN-07 – Missing Security Headers – Low: Common browser security headers were not explicitly configured. Recommendation: add appropriate headers and a Content Security Policy.

VULN-08 – Lack of Login Rate Limiting – Medium: Authentication requests were not rate limited. Recommendation: add rate limiting and monitoring around login endpoints.

8. Secure Coding Improvements

The secure version implemented environment-based secret configuration, parameterized SQL queries, password hashing and `check_password_hash()`, input validation, safer session cookie configuration, security response headers, a Content Security Policy, disabled Flask debug mode, session clearing during login, and authorization checks for note deletion.

9. Bandit AFTER Results

After remediation, Bandit reported: "No issues identified." The after-scan reports were saved as `bandit-after.txt` and `bandit-after.json`. Total reported issues were 0 for High, Medium and Low severity.

10. Before vs After

Secret key: hardcoded → environment-based configuration

SQL queries: string construction → parameterized queries

Passwords: plaintext → hashed

Debug mode: enabled → disabled

Input handling: limited → validated

Security headers: absent → configured

Bandit findings: 3 → 0

11. Functional Testing

The secure application was tested locally. Registration, login, authenticated dashboard access, note creation, note display, note deletion and logout were tested. The application remained functional after the security improvements.

12. Recommended Secure Coding Practices

Never commit secrets or passwords to source control. Use parameterized queries. Hash passwords. Disable debug mode in production. Validate all user-controlled input. Use CSRF protection for state-changing forms. Use HTTPS and security headers in production. Apply authentication rate limiting. Keep dependencies updated and repeat security analysis regularly.

13. Conclusion

The review identified several common web-application security weaknesses. Bandit detected the hardcoded secret, unsafe SQL construction and enabled debug mode, while manual review identified additional issues such as plaintext password storage, missing input validation, CSRF protection, security headers and login rate limiting. The secure version applied recommended coding practices and the final Bandit scan reported no identified issues.

14. Project Evidence

reports/bandit-before.txt
reports/bandit-before.json
reports/bandit-after.txt
reports/bandit-after.json
screenshots/
vulnerable_app/
vulnerable_app_original/
secure_app/
README.md
.gitignore

15. References

Bandit Python Security Linter documentation.
Flask documentation.
Werkzeug security and password-hashing documentation.
OWASP Secure Coding Practices and OWASP Top 10 guidance.

16. Screenshot Evidence

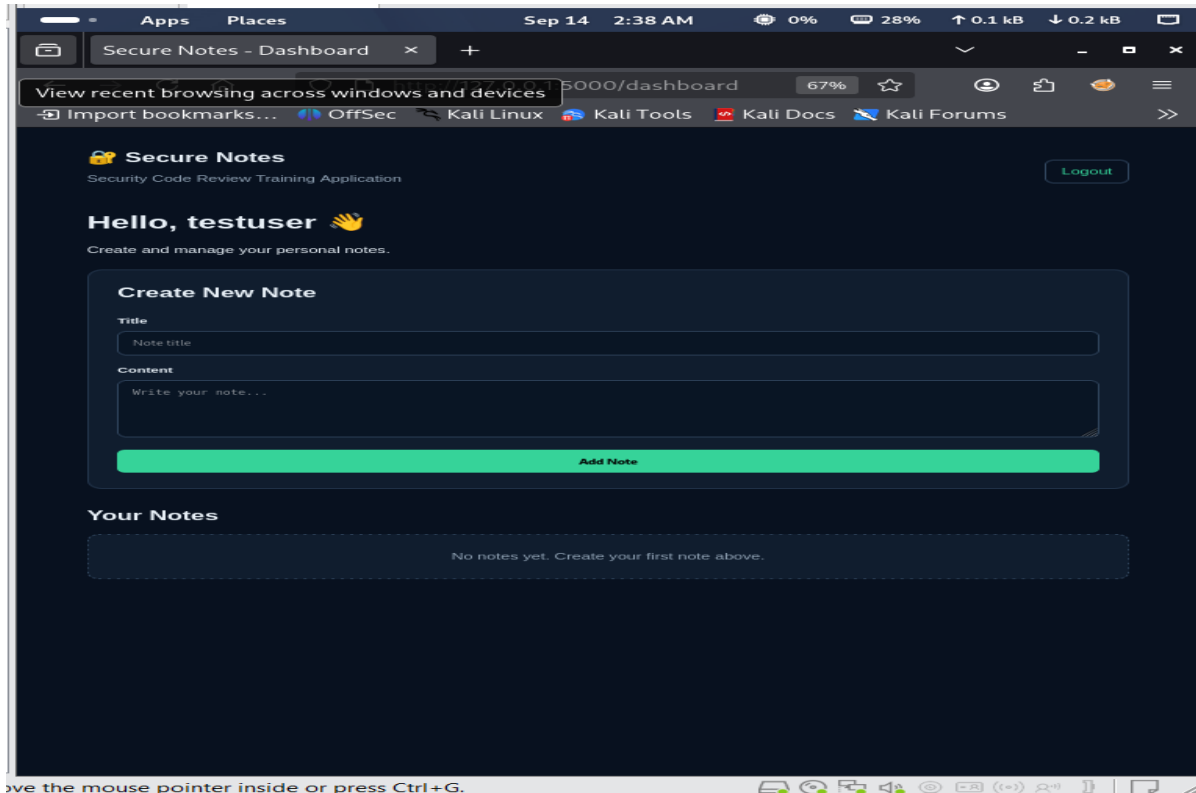


Figure 1: Project evidence.

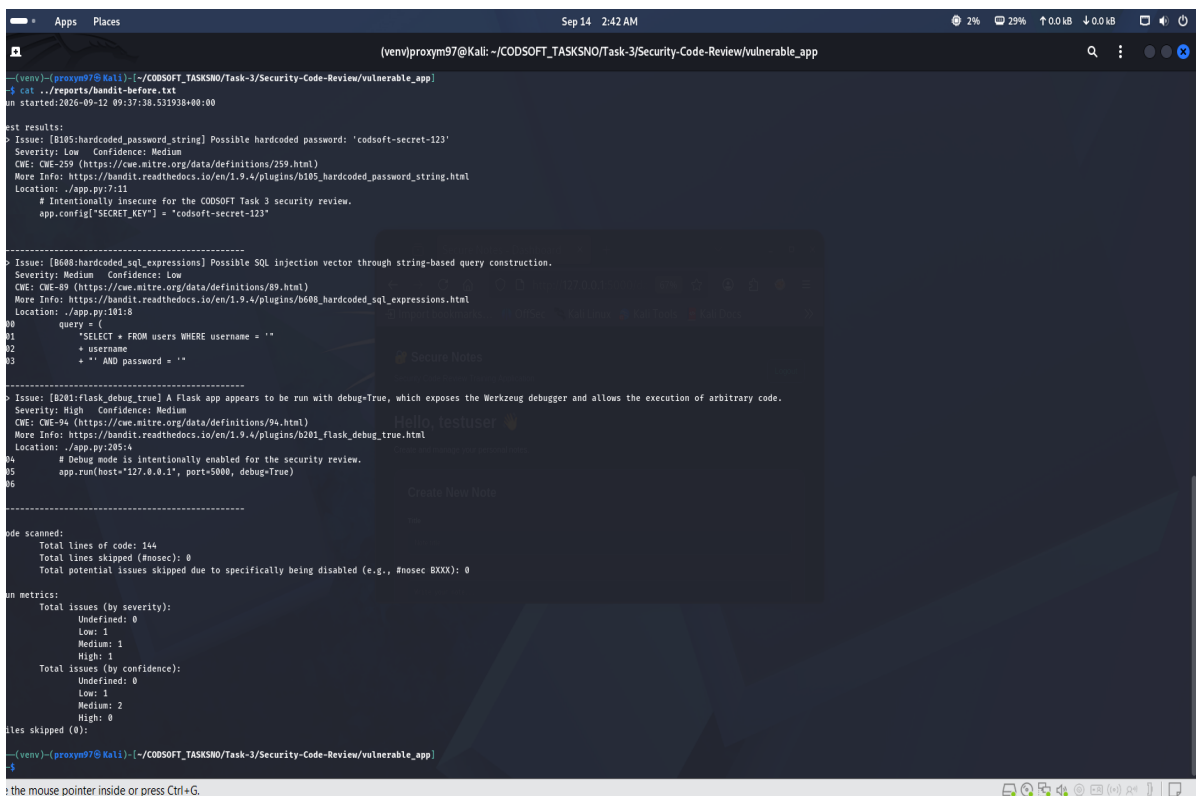


Figure 2: Project evidence.

```
(proxym97@Kali)-[~/CODSOFT_TASKSNO/Task-3/Security-Code-Review/vulnerable_app]
$ cd ~/CODSOFT_TASKSNO/Task-3/Security-Code-Review/vulnerable_app_original

(proxym97@Kali)-[~/CODSOFT_TASKSNO/Task-3/Security-Code-Review/vulnerable_app_original]
$ grep -n -A12 -B3 "SELECT \* FROM users" app.py
150-     conn = get_db()
151-
152-     query = """
153:     ← SELECT \* FROM users
154-         WHERE username = ?
155-     """
156-
157-     user = conn.execute(
158-         query,
159-         (username,)
160-     ).fetchone()
161-
162-     conn.close()
163-
164-     if user and check_password_hash(user["password"], password):
165-         session.clear()
```

Figure 3: Project evidence.

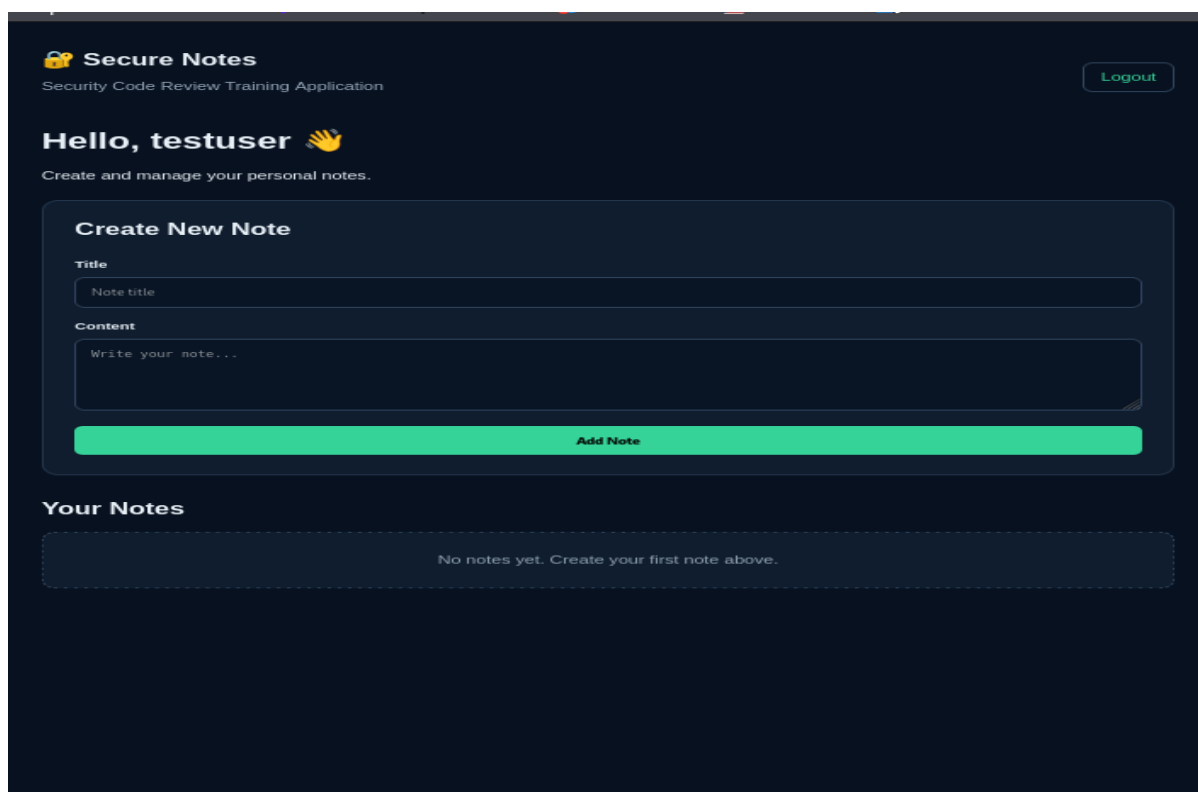


Figure 4: Project evidence.


```

(venv)-(proxym97@Kali)-[~/CODSOFT_TASKSNO/Task-3/Security-Code-Review/secure_app]
Sgrep -n -A5 -B5 "check_password_hash" app.py
mand 'Sgrep' not found, but there are 18 similar ones.

(venv)-(proxym97@Kali)-[~/CODSOFT_TASKSNO/Task-3/Security-Code-Review/secure_app]
grep -n -A5 -B5 "check_password_hash" app.py
import os
import sqlite3

from flask import Flask, render_template, request, redirect, url_for, session, flash
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)

Secret should be supplied through the environment.
app.config["SECRET_KEY"] = os.environ.get(

    (username,)
    ).fetchone()

conn.close()

if user and check_password_hash(user["password"], password):
    session.clear()
    session["username"] = user["username"]

    return redirect(url_for("dashboard"))

```

Figure 7: Project evidence.